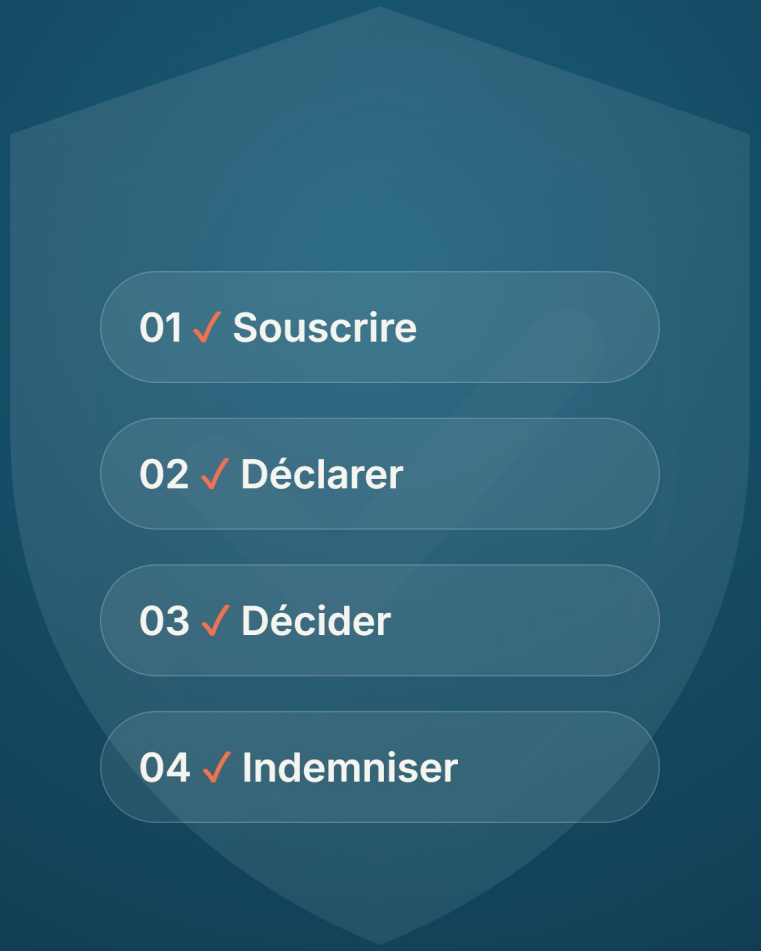
InsurSmart

L'assurance qui vous suit, étape par étape.

Plateforme de gestion des assurances en microservices ·
Présentation du projet et modélisation

Jendoubi Leila · Ben Amara Hiba · Hchicha Adem · Jouini Aziz

ESPRIT · MODULE AWD · VALIDATION 1 · 2026-2027



Le dossier d'Amira, avant InsurSmart

PERSONA FICTIVE

Amira, 29 ans

Assurée auto à Tunis. Un léger accrochage, et commence le parcours du combattant.

Déclarer prend du temps

Appels, e-mails, papier : elle répète trois fois la même histoire.

Aucune visibilité

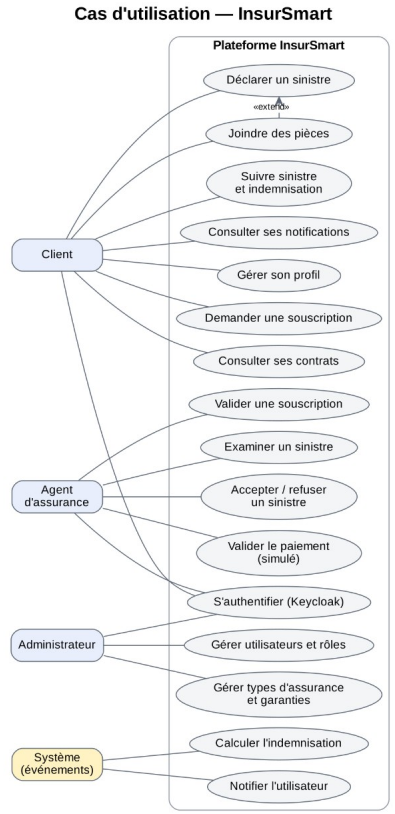
Où en est son dossier ? Personne ne peut le lui dire tout de suite.

Un calcul opaque

Franchise, plafond : elle ne comprend pas d'où vient le montant.

Comment rendre ce parcours simple, transparent et traçable, avec une architecture où chaque métier évolue indépendamment ?

Trois acteurs humains, un système qui veille



Client · CLIENT
Souscrit, déclare, suit son dossier

Agent d'assurance · AGENT
Valide, examine, décide, paie

Administrateur · ADMIN
Catalogue, garanties, utilisateurs

Système · événements
Calcule et notifie

Le parcours, de la souscription au paiement

1. Souscription validée
2. Consultation des contrats
3. Déclaration avec pièces
4. Éligibilité vérifiée
5. Examen et décision
6. Calcul automatique
7. Notification et timeline

LA RÈGLE DE CALCUL

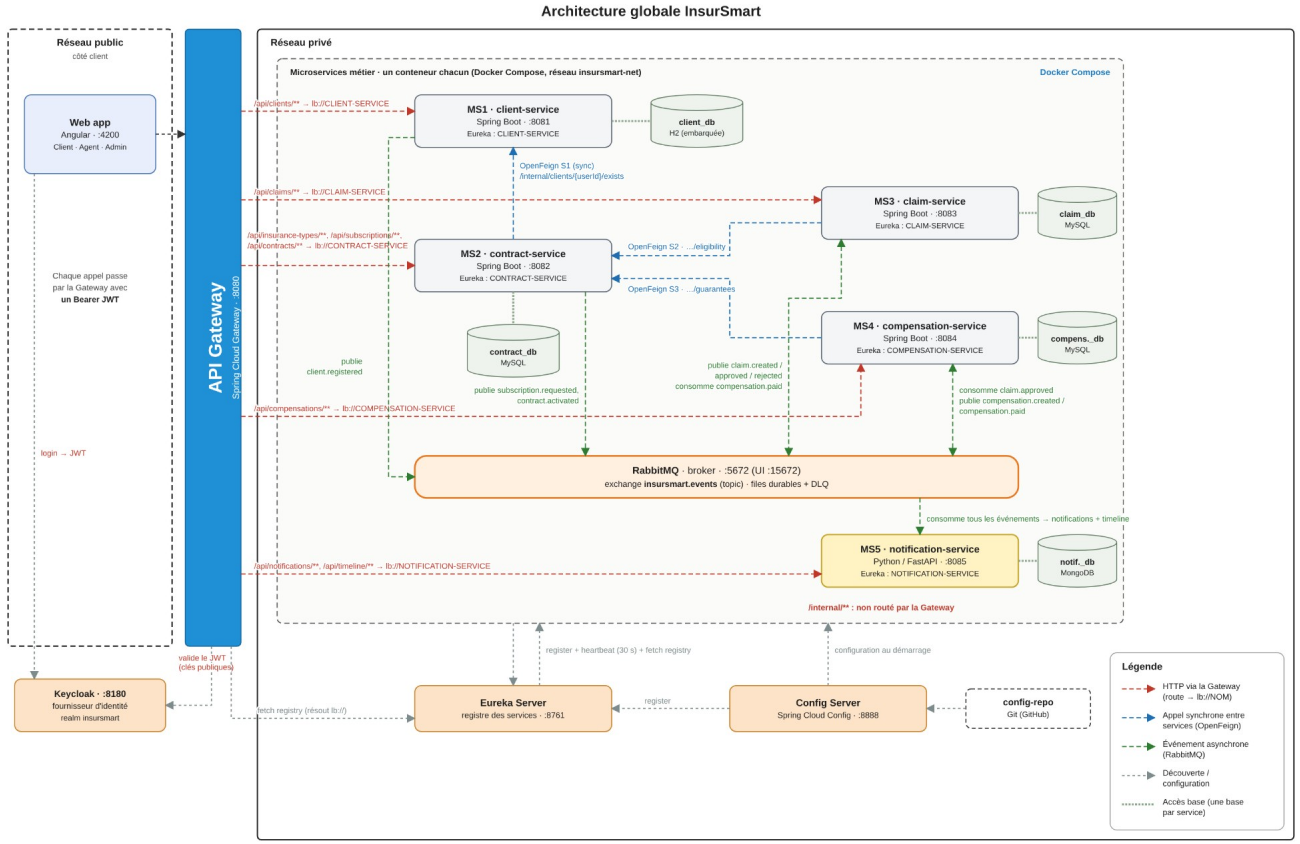
montant = min(max(0, réclamé − franchise), plafond)

✓ Sinistre seulement sur un contrat ACTIVE du client

✓ Une seule indemnisation par sinistre

✓ userId = identité Keycloak, partout

Une seule porte d'entrée, une base par service



Cinq services, cinq métiers, cinq bases

client
-service

Profils, adresse, préférences

Spring Boot · 8081

H2

[Étudiant 1]

contract
-service

Catalogue, souscriptions, contrats

Spring Boot · 8082

MySQL

[Étudiant 2]

claim
-service

Sinistres, pièces, décision

Spring Boot · 8083

MySQL

[Étudiant 3]

compensation
-service

Calcul et paiement simulé

Spring Boot · 8084

MySQL

[Étudiant 4]

notification
-service

Notifications, timeline

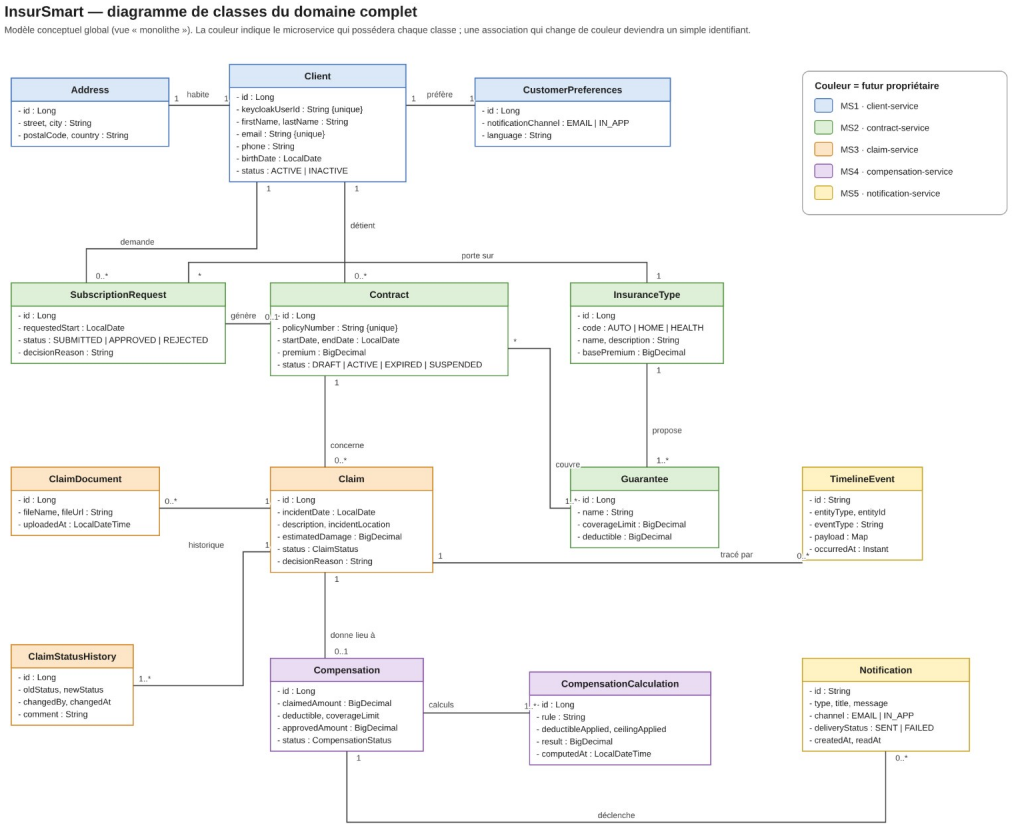
FastAPI · 8085

MongoDB

Toute l'équipe

claim décide, compensation calcule. Chaque base MySQL tourne dans son propre conteneur ; aucune n'est partagée.

Du modèle global aux contextes bornés



1 **Un modèle global**

Toutes les classes, vue « monolithe ». La couleur = le futur propriétaire.

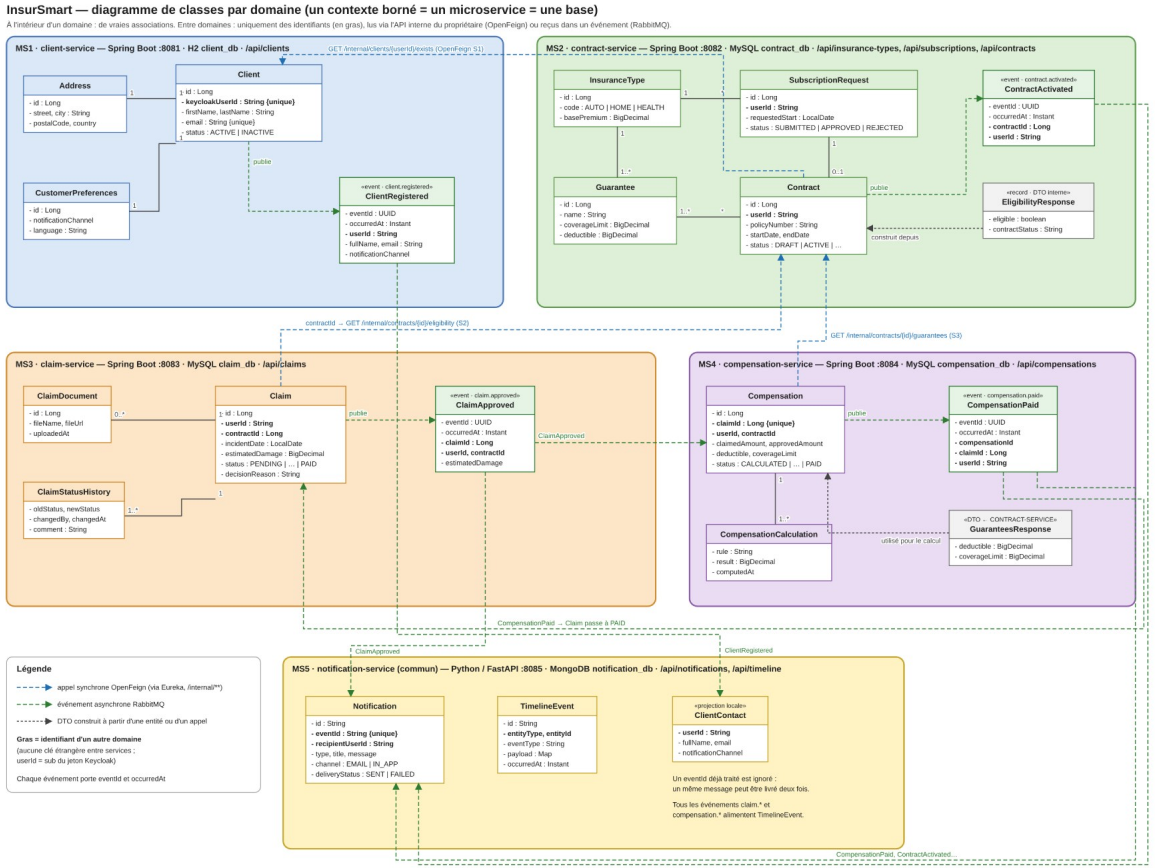
2 **On coupe aux frontières**

Une association qui change de couleur devient un simple identifiant.

3 **Zéro clé étrangère**

La donnée externe arrive par OpenFeign ou par un événement.

Entre domaines, seulement des identifiants



Synchrone pour décider, asynchrone pour informer

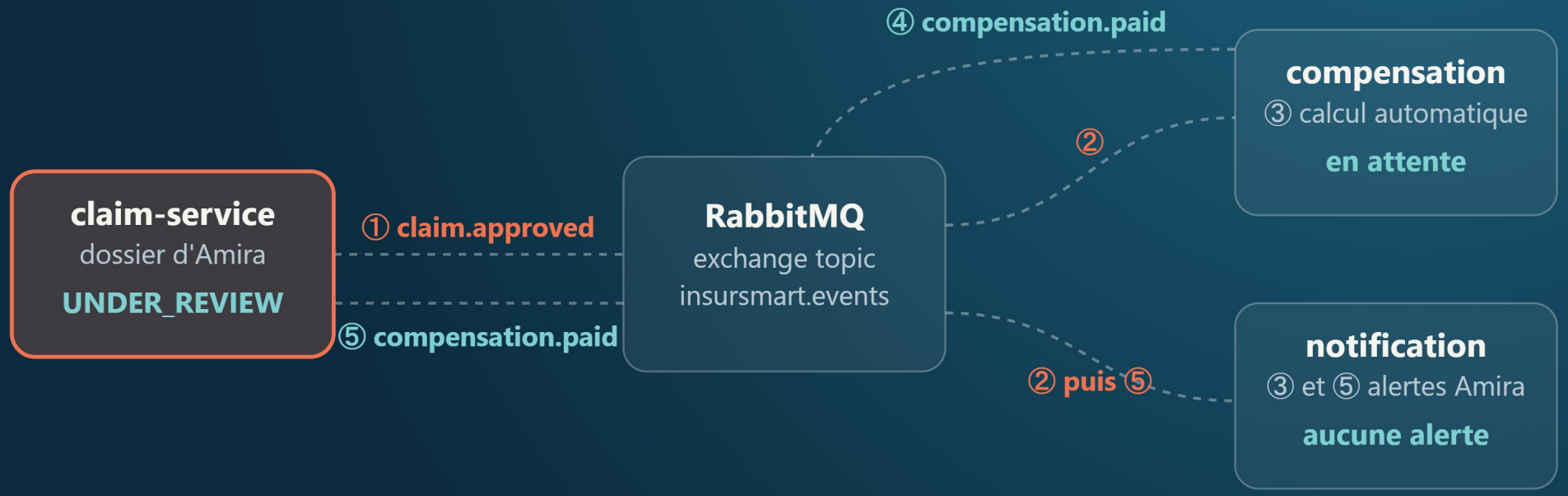
OpenFeign · /internal/**

Appel	Si le service tombe
contract → client	503 : pas de souscription
claim → contract	503 : pas de sinistre
compensation → contract	retry, puis DLQ

RabbitMQ · insursmart.events

Événement	Consommateurs
claim.created	notification
claim.approved	compensation, notification
compensation.paid	claim, notification
client.registered...	notification

Un événement, plusieurs réactions — en direct



① claim.approved
claim-service publie : l'agent a accepté

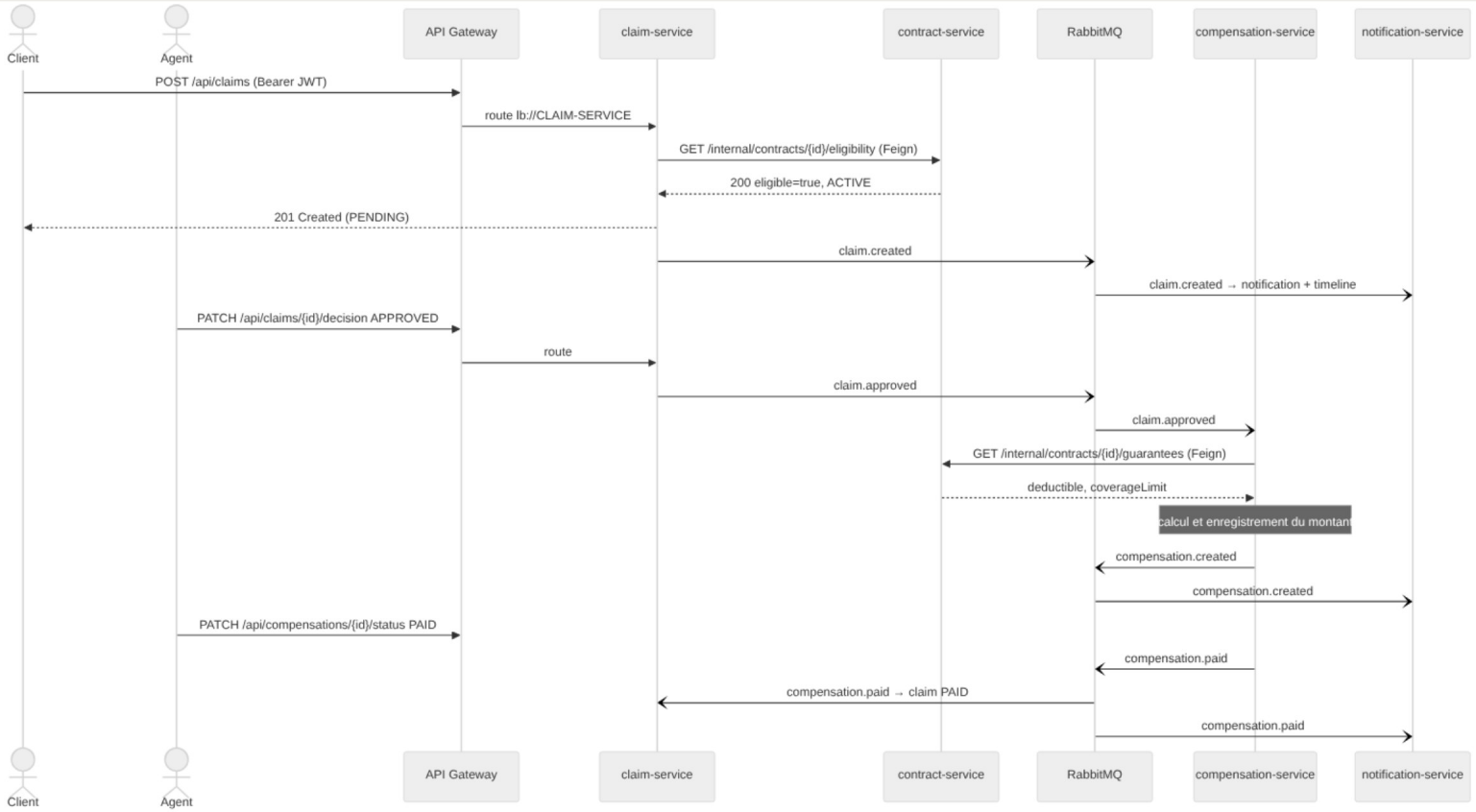
② Distribution
RabbitMQ copie l'événement dans 2 files

③ Réactions
Calcul de 1 600 · alerte « Sinistre accepté »

④ compensation.paid
compensation-service publie le paiement

⑤ Clôture
Sinistre PAID · alerte « Paiement reçu »

Le dossier d'Amira, de bout en bout



Le jeton est vérifié à chaque étage



ÉTAPE 1

Connexion

Authorization Code + PKCE : Keycloak délivre un JWT avec les rôles



ÉTAPE 2

Gateway

Signature vérifiée avec les clés JWKS, sinon 401



ÉTAPE 3

Service

Rôle et propriété : userId du jeton = userId de la ressource, sinon 403



ÉTAPE 4

Interne

/internal/** jamais routé ; Feign propage le jeton

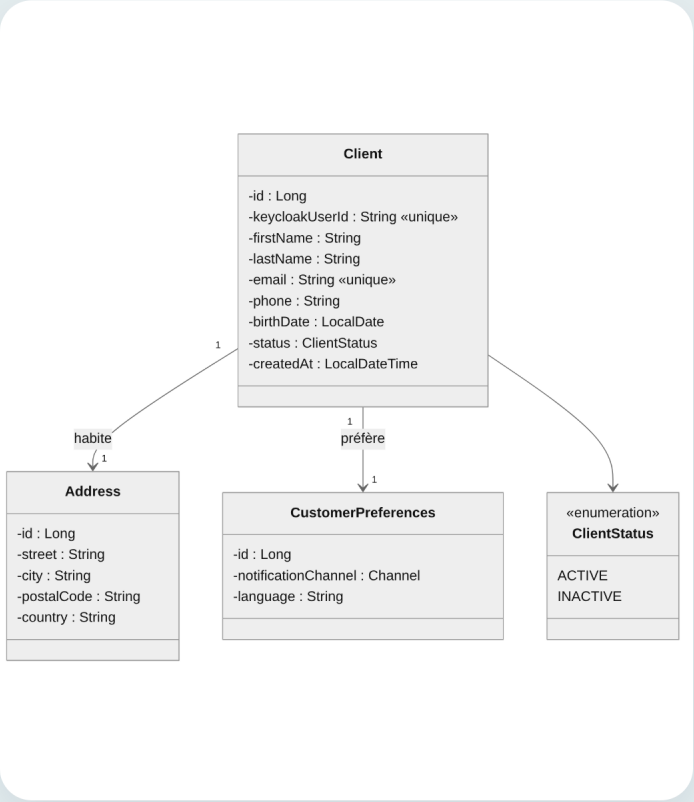
Rôles **CLIENT** · **AGENT** · **ADMIN** — realm exporté en JSON dans Git, bases jamais exposées.

La stack imposée, plus un service Python

Frontend	Angular · keycloak-angular
Entrée et sécurité	Spring Cloud Gateway · Keycloak (OAuth2 / OIDC)
Microservices	Spring Boot 3 · Java 17 + Python 3.12 · FastAPI
Infrastructure	Eureka · Config Server · OpenFeign · Resilience4j · RabbitMQ
Données et déploiement	H2 · MySQL · MongoDB · Docker Compose

MS1 · [ÉTUDIANT 1]

client-service, le profil d'Amira



Spring Boot · 8081 | H2 | CLIENT-SERVICE

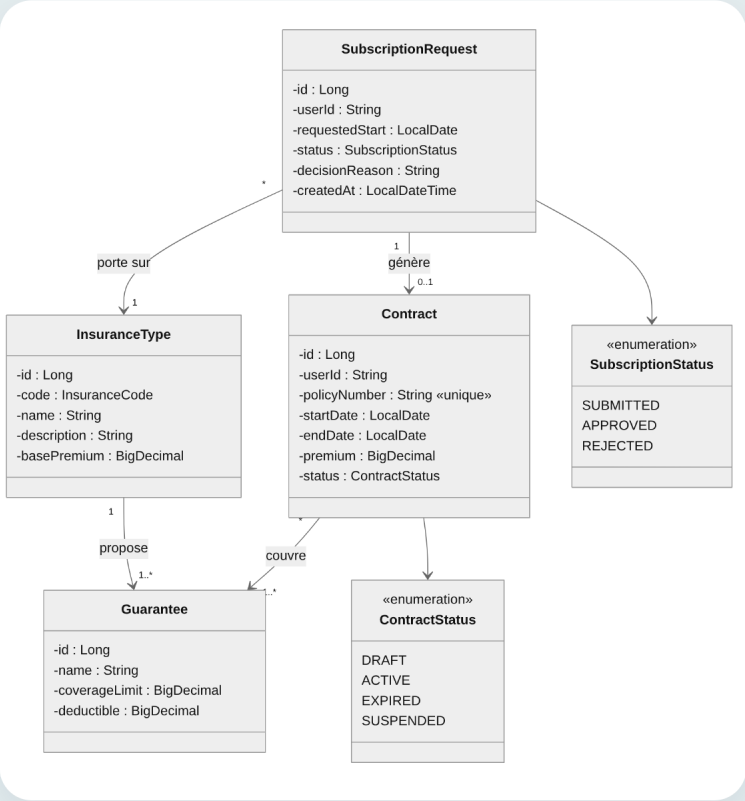
Endpoint	Codes
POST /api/clients	201 · 400 · 409
GET /api/clients/me	200 · 401
PUT /api/clients/{id}	200 · 403 · 404
PUT /api/clients/me/preferences	200 · 400
GET /internal/clients/{userId}/exists	200 · 404

Interactions · appelé par contract-service (S1) ; publie client.registered, client.updated

Si panne · aucune dépendance sortante

MS2 · [ÉTUDIANT 2]

contract-service, la vérité sur la couverture



Spring Boot · 8082 | MySQL | CONTRACT-SERVICE

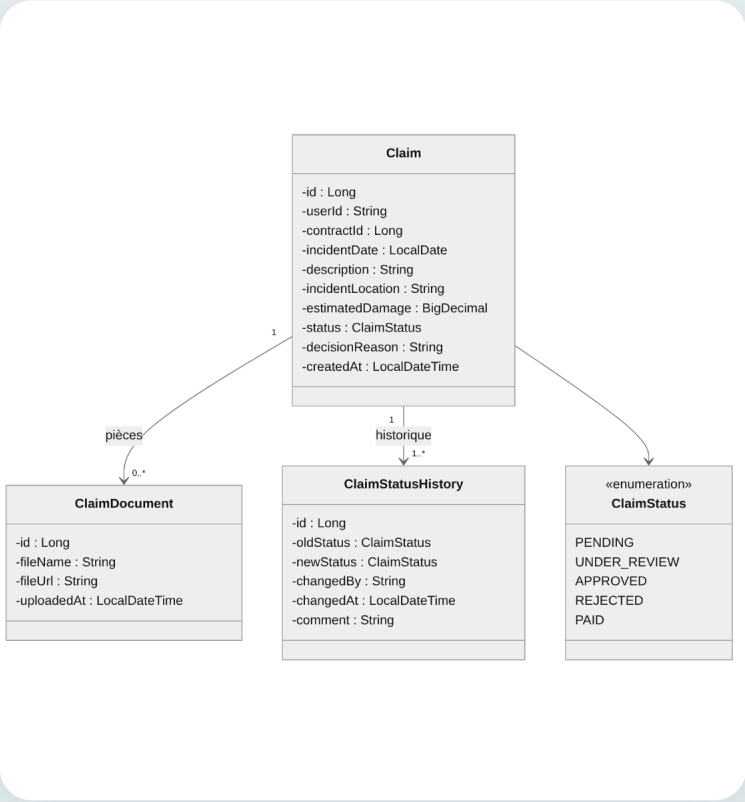
Endpoint	Codes
POST /api/subscriptions	201 · 404 · 503
PATCH /api/subscriptions/{id}/decision	200 · 403 · 409
GET /api/contracts/{id}	200 · 403 · 404
GET /internal/contracts/{id}/eligibility	200 · 404
GET /internal/contracts/{id}/guarantees	200 · 404

Interactions · appelle client (S1) ; appelé par claim (S2) et compensation (S3) ; publie contract.activated

Si panne · client-service muet : 503, aucune souscription

MS3 · [ÉTUDIANT 3]

claim-service, la déclaration et la décision



Spring Boot · 8083 | MySQL | CLAIM-SERVICE

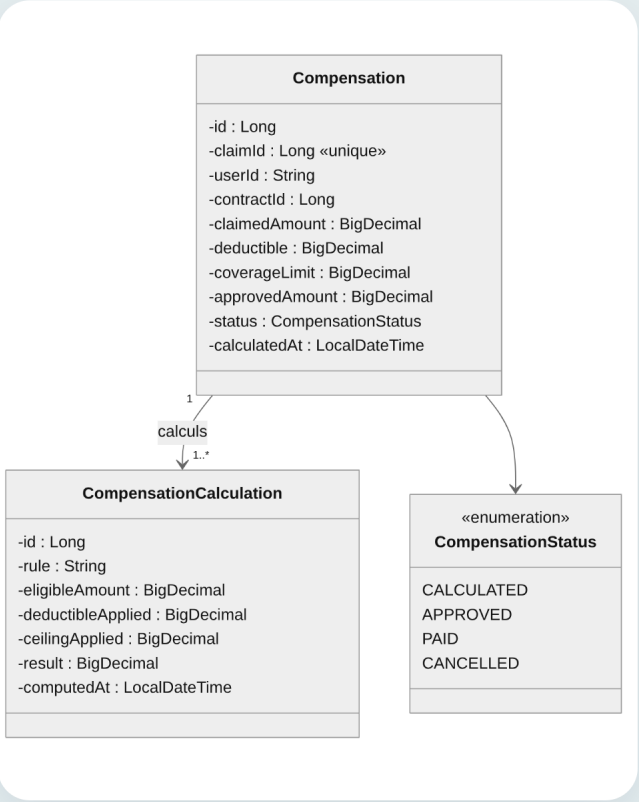
Endpoint	Codes
POST /api/claims	201 · 404 · 409 · 503
POST /api/claims/{id}/documents	201 · 413
PATCH /api/claims/{id}/review	200 · 409
PATCH /api/claims/{id}/decision	200 · 403 · 409
GET /api/claims/{id}/history	200 · 404

Interactions · appelle contract (S2) ; publie claim.created, .approved, .rejected ; consomme compensation.paid

Si panne · contract-service muet : 503, jamais de sinistre non vérifié

MS4 · [ÉTUDIANT 4]

compensation-service, le calcul automatique



Spring Boot · 8084 | MySQL | COMPENSATION-SERVICE

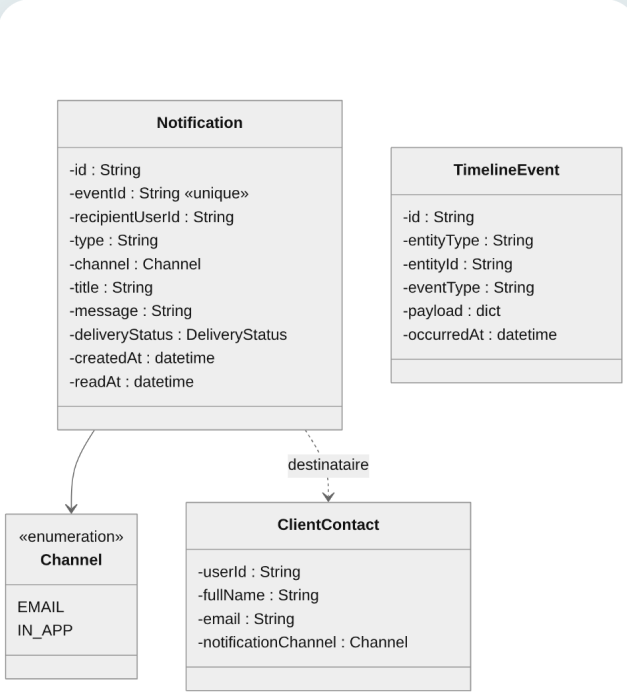
Endpoint	Codes
GET /api/compensations/claim/{claimId}	200 · 404
POST /api/compensations/{id}/recalculate	200 · 409
PATCH /api/compensations/{id}/status	200 · 400 · 409
GET /api/compensations/{id}/calculations	200 · 404

Interactions · consomme claim.approved ; appelle contract (S3) ; publie compensation.created, .paid

Si panne · calcul reporté : retry, puis DLQ

MS5 COMMUN · PYTHON / FASTAPI

notification-service, la voix de la plateforme



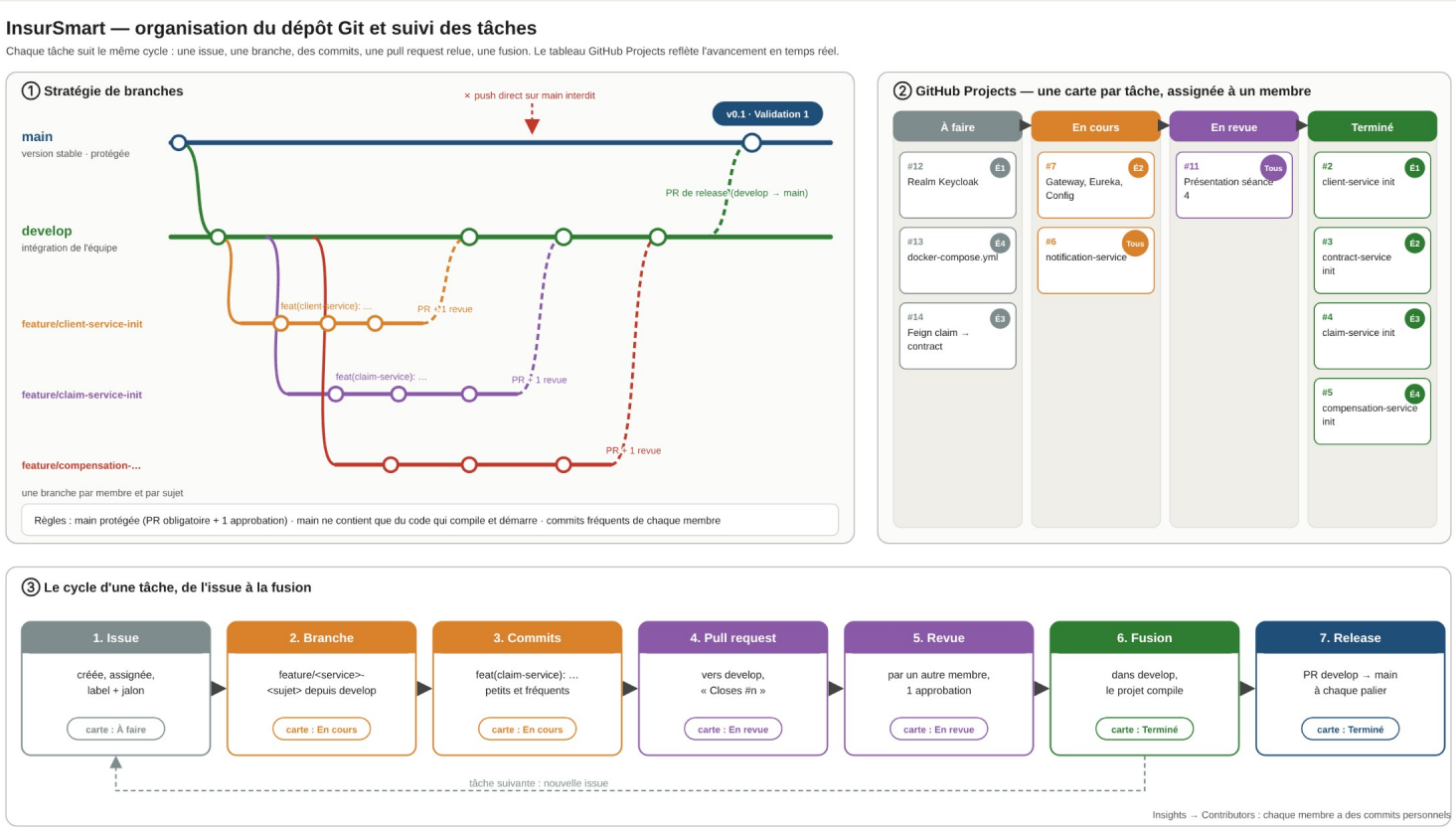
FastAPI · 8085 | MongoDB | NOTIFICATION-SERVICE

Endpoint	Codes
GET /api/notifications/me	200 · 401
GET /api/notifications/me/unread-count	200
PATCH /api/notifications/{id}/read	200 · 403 · 404
GET /api/timeline/claims/{claimId}	200 · 403 · 404

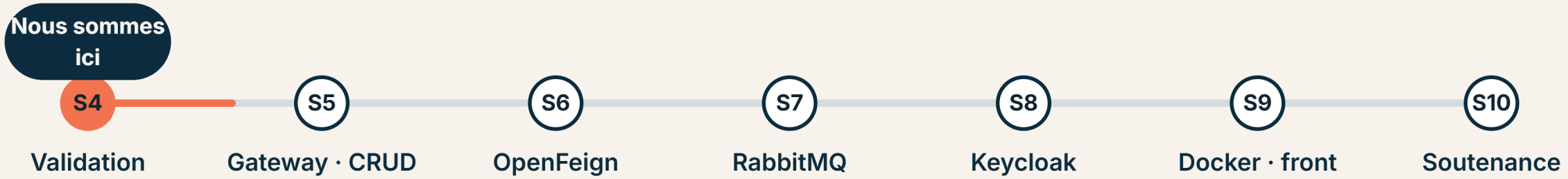
Interactions · consomme tous les événements (aio-pika) ; enregistré dans Eureka ; Swagger /swagger-ui

Si panne · rien n'est bloqué : les messages attendent dans RabbitMQ

Une issue, une branche, une pull request relue



Un palier d'intégration par séance



RISQUE

Contrats d'API divergents

→ Figés dès S4, mocks

RISQUE

Service appelé en panne

→ Circuit breaker, 503

RISQUE

Message perdu ou en double

→ ACK, DLQ, eventId

RISQUE

Deux langages backend

→ Conventions communes

Merci.

Le dossier d'Amira est clos : indemnisée, notifiée, tracée.

github.com/Hibabenamara/InsurSmart-4Actuariat-g4